

Smart Car Rental Booking System using Queue, Stack, and Greedy Algorithm

PROJECT REPORT

On

Smart Car Rental Booking System

Submitted in partial fulfillment of the requirements for the award of the
degree of Bachelor of Technology in Computer Science and Engineering

Submitted by:

D. Mahesh Kumar --- AP24110011115

M. Bhavana Reddy — AP24110011101

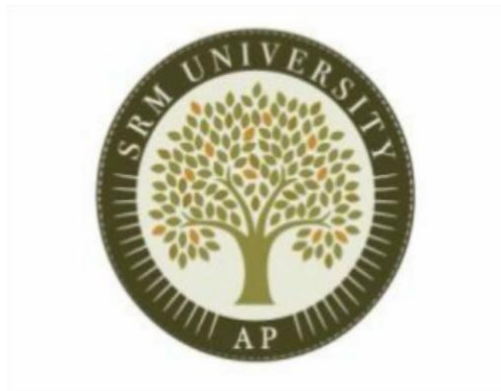
V. HimaSri — AP24110011096

P. Sri Sindhu — AP24110011070

Under the Guidance of:

SURIYA

**Department of CSE Department of Computer Science and Engineering SRM
University, Andhra Pradesh**



Academic Year:2025-2026

Abstract

This project presents a full-stack Car Rental Booking System designed to efficiently manage vehicle reservations using fundamental data structures and algorithms. The system utilizes a Queue to handle booking requests in a first-come, first-served manner, ensuring fairness, while a Stack is implemented to manage admin operations with undo functionality for better control and flexibility. Additionally, a Greedy Algorithm is applied to allocate the most suitable available car to customers based on predefined criteria such as availability and cost efficiency.

The frontend is developed using React, providing an interactive and user-friendly interface enhanced with immersive 3D visualizations for an improved user experience. The backend is powered by Node.js and MongoDB, ensuring scalable data management and seamless communication between components. Overall, the system demonstrates how core computer science concepts can be effectively applied to build a practical, efficient, and user-centric application.

Introduction

Car rental systems provide convenient transportation, but managing bookings and allocating vehicles efficiently can be challenging, especially during high demand. Traditional systems may cause delays and inefficient resource usage.

This project introduces a Smart Car Rental Booking System that uses data structures and algorithms to improve performance. A Queue (FIFO) ensures fair processing of booking requests, while a Stack (LIFO) allows admin actions to be undone easily. A Greedy Algorithm is used to assign the most suitable car based on factors like price and availability.

The system is developed as a full-stack application using React.js for the frontend and Node.js with MongoDB for the backend, ensuring a responsive and efficient user experience.

Objectives

The main objective of this project is to develop an efficient car rental booking system using data structures and algorithms for better request handling and resource allocation. The key objectives are:

- To design a scalable system capable of handling multiple booking requests efficiently.
- To implement a Queue-based system for fair and systematic booking processing.
- To use a Stack for admin operations, enabling undo functionality.
- To apply a Greedy Algorithm for optimal car assignment based on user constraints.
- To develop a responsive and interactive frontend using React.js.
- To ensure secure data handling using MongoDB and efficient backend processing with Node.js.
- To provide a smooth and user-friendly experience with real-time updates.
- To allow future enhancements such as AI recommendations and payment integration.

System Architecture

Frontend: React.js, Three.js, TailwindCSS

Backend: Node.js, Express.js

Database: MongoDB

Communication: REST APIs (Axios)

FRONTEND AND BACKEND FILE DESCRIPTION

1) Frontend Files Description

The frontend of the Smart Car Rental System is developed using React, Vite, and Tailwind CSS. The frontend

provides a modern, responsive, and user-friendly interface for vehicle browsing and booking operations.

i. Frontend Folder Structure

Client/

|

|— node_modules/

|— public/

|— src/

|— .gitignore

|— eslint.config.js

|— index.html

```
graph LR; Root[ ] --- package.json; Root --- package-lock.json; Root --- vite.config.js; Root --- README.md;
```

|— package.json
— package-lock.json
|— vite.config.js
└— README.md

ii. node_modules Folder

The node_modules folder contains all installed frontend dependencies and external libraries required for the project. This folder is automatically generated when the npm install command is executed.

The folder contains libraries such as:

1. React
2. Axios
3. Tailwind CSS
4. React Router DOM
5. Framer Motion
6. React Hot Toast

This folder is ignored using the .gitignore file because of its very large size.

iii. public Folder

The public folder stores static assets used in the frontend application.

It contains:

1. Images
2. Logos
3. Icons
4. Static assets
5. Favicon files

Files inside the public folder are directly accessible by the browser and are not processed by React.

iv. src Folder

The src folder contains the complete React application source code.

This is the most important frontend folder because it contains:

1. Components
2. Pages
3. Routing logic
4. API communication
5. Utility functions
6. Styling files

All React development and application rendering start from this folder.

v. .gitignore File

The .gitignore file is used to prevent unnecessary files from being uploaded to GitHub.

Examples of ignored files include:

1. node_modules
2. dist folder
3. log files
4. temporary files

This helps maintain a clean and lightweight repository.

vi. eslint.config.js File

The eslint.config.js file is used for maintaining clean and error-free code.

Main purposes include:

1. Detecting unused variables
2. Maintaining coding standards
3. Improving code readability
4. Reducing coding mistakes

ESLint improves the maintainability and quality of the frontend application.

vii. index.html File

The index.html file is the main HTML entry point of the frontend application.

It contains the root div where the complete React application is rendered.

Example:

```
<div id="root"></div>
```

The React application is loaded using:

```
<script type="module" src="/src/main.jsx"></script>
```

This file acts as the starting point of the frontend application.

viii. package.json File

The package.json file contains:

1. Project metadata
2. Dependencies
3. Scripts
4. Version information

Important frontend scripts include:

```
"dev": "vite"
```

This command starts the development server.

```
"build": "vite build"
```

This command creates the production build.

Main frontend dependencies include:

5. React
6. Axios
7. Tailwind CSS
8. React Router DOM
9. Framer Motion
10. React Hot Toast

ix. package-lock.json File

The package-lock.json file stores the exact versions of all installed dependencies.

Advantages include:

1. Stable installations
2. Dependency consistency
3. Reliable project setup

This file is automatically generated by npm.

x. vite.config.js File

The vite.config.js file contains Vite configuration settings.

The uploaded project contains the following configuration:

```
plugins: [  
  react(),  
  tailwindcss(),
```

]

This configuration enables:

1. React integration
2. Tailwind CSS support
3. Faster builds
4. Hot module replacement

Vite improves frontend performance and development speed.

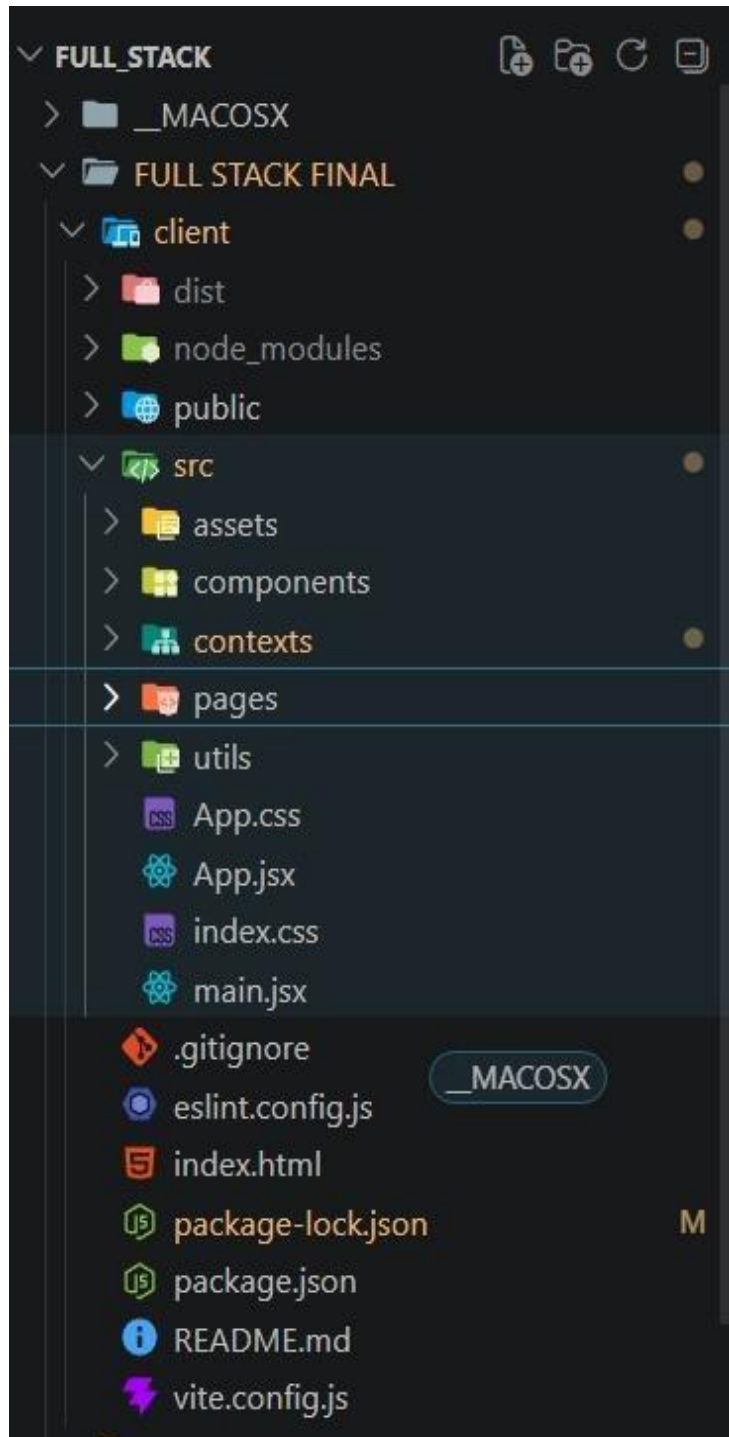
xi. README.md File

The README.md file contains frontend project documentation.

It explains:

1. Project overview
2. Installation process
3. Technologies used
4. Setup instructions
5. Project features

This file helps developers understand and run the project easily.



FRONTEND LINK:

https://drive.google.com/file/d/16x7_NC2mmjMta_GmNbC_R723Xo9cnWiL/view?usp=drive_link

1) BACKEND FILES DESCRIPTION

The backend of the Smart Car Rental System is developed using Node.js, Express.js, and MongoDB. The backend handles authentication, APIs, booking management, database operations, and security. The system uses MongoDB as the database, which stores information such as user details, vehicle availability, booking records, and transaction history. The integration of MongoDB allows efficient handling of large datasets with flexible schema design. To manage booking requests efficiently, a Queue data structure is implemented, ensuring that customer requests are processed in a First-In-First-Out (FIFO) manner. This helps in maintaining fairness and avoiding conflicts during high-demand scenarios. An admin action management system is implemented using a Stack, enabling undo functionality for operations such as booking cancellations or vehicle updates. This enhances system reliability and error recovery.

i. Backend Folder Structure

```
|— node_modules/  
|— config/  
|— controllers/  
|— middleware/  
|— models/  
|— routes/  
|— .env  
|— package.json  
|— package-lock.json  
|— seed.js  
└— server.js
```

ii. node_modules Folder

The node_modules folder contains all installed backend dependencies and libraries.

It is automatically generated after running:

```
npm install
```

The folder contains libraries such as:

1. Express.js
2. Mongoose
3. JWT
4. bcryptjs
5. Cloudinary
6. Multer

iii. config Folder

The config folder contains configuration-related files.

Its main purpose is:

1. MongoDB database connection setup
2. Environment configuration

Separating configuration logic improves project organization and maintainability.

iv. controllers Folder

The controllers folder contains the business logic of the application.

Responsibilities include:

1. User authentication
2. Vehicle management
3. Booking management
4. Request processing

Controllers receive requests from routes and send responses back to the frontend.

v. middleware Folder

The middleware folder contains reusable functions executed before route handlers.

Examples include:

1. Authentication middleware

2. Error handling middleware
3. Validation middleware

Middleware improves:

1. Security
2. Validation
3. Code reusability

vi. models Folder

The models folder contains MongoDB schemas created using Mongoose.

Models define:

1. Database collections
2. Data structure
3. Validation rules
4. Collection fields

Examples include:

1. User model
2. Vehicle model
3. Booking model

vii. routes Folder

The routes folder contains API route definitions.

Examples include:

1. Authentication routes
2. Vehicle routes
3. Booking routes
4. User routes

5. Dashboard routes

Routes connect frontend requests to backend controllers.

viii. .env File

The .env file stores sensitive environment variables securely.

Example:

PORT=5001

MONGO_URI=mongodb://localhost:27017/mydb

JWT_SECRET=supersecretjwtkeyforfutureride

Main purposes include:

1. Database connection setup
2. Port configuration
3. JWT secret storage

This improves application security.

ix. package.json File

The backend package.json file contains:

1. Dependencies
2. Scripts
3. Project metadata

Important scripts include:

"start": "node server.js"

Starts the backend server.

"dev": "nodemon server.js"

Runs the backend using nodemon for automatic restart during development.

Main backend dependencies include:

4. Express.js
5. MongoDB
6. Mongoose
7. JWT
8. bcryptjs
9. Cloudinary
10. Multer
11. dotenv

x. package-lock.json File

The package-lock.json file stores the exact versions of installed backend dependencies.

Advantages include:

1. Stable installations
2. Dependency consistency
3. Reliable project setup

This file is automatically generated by npm.

xi. seed.js File

The seed.js file is used to insert sample vehicle data into MongoDB.

The dataset contains:

1. Electric vehicles
2. Luxury vehicles
3. SUVs

4. Sedans

Vehicle examples include:

5. Tesla Model S Plaid
6. Audi e-tron GT
7. Bentley Continental GT
8. Rolls Royce Neon Phantom

Each vehicle contains:

9. Vehicle name
10. Brand
11. Price
12. Fuel type
13. Images
14. Ratings
15. Transmission type

This file helps initialize the database quickly with sample vehicle data.

xii. server.js File

The server.js file is the main backend entry point of the application.

Main responsibilities include:

1. Creating Express server
2. Connecting MongoDB database
3. Configuring middleware
4. Registering API routes
5. Configuring error handling
6. Starting backend server

Main API routes include:

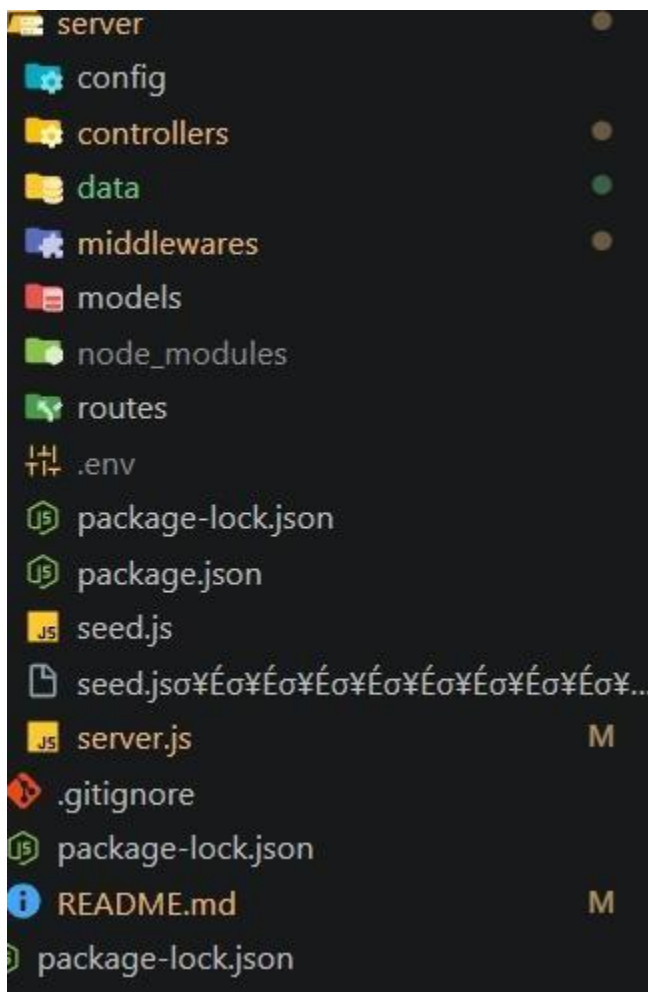
7. /api/auth
8. /api/vehicles
9. /api/bookings
10. /api/users
11. /api/dashb

oard The file also

includes:

12. CORS configuration
13. JSON middleware
14. Health check API
15. Express rate limiting

This file acts as the central controller of the backend application.



BACKEND LINK:

https://drive.google.com/file/d/1ov_bVYZJ9MGOG259CK7Q7UwN7Eh-NfB2/view?usp=drive_link

Data Structures Used :

a. Queue (FIFO)

Used for booking requests:

- New bookings are added to the rear.
- Processed from the front.

❖ Stack (LIFO)

Used for admin history:

- Actions pushed on top.
- Undo removes last action.

Greedy Algorithm (Car Assignment)

Logic:

Filter available cars

Sort by:

Price (ascending)

Distance (nearest first)

Select the best car

Sample Code (JavaScript)

```
function assignCar(cars, request) {  
    // Step 1: Filter available cars  
    let availableCars = cars.filter(car => car.availability_status ===  
    "available");  
  
    // Step 2: Sort (Greedy choice)  
    availableCars.sort((a, b) => {  
        if (a.price_per_day !== b.price_per_day) {  
            return a.price_per_day - b.price_per_day;  
        }  
        return a.location.localeCompare(b.location);  
    });
```

// Step 3: Select best car

```
for (let car of availableCars) {  
    if (car.price_per_day <= request.preferred_budget) {  
        car.availability_status = "booked";  
        return car;  
    }  
}  
  
return "No cars available";  
}
```

Time Complexity

Filtering: $O(n)$

Sorting: $O(n \log n)$

Selection: $O(n)$

Overall: $O(n \log n)$

Sample Input

Cars:

```
[  
  { "id":1, "price":1000, "location":"A", "availability":"available" },  
  { "id":2, "price":800, "location":"B", "availability":"available" }  
]
```

Request:

```
{ "budget":900, "location":"B" }
```

Sample Output

Selected Car:

```
{ "id":2, "price":800, "location":"B" }
```


Features

- Real-time booking queue for systematic request processing.
- Admin undo functionality using stack operations.
- 3D visualization of data structures for better understanding.
- Secure authentication for user and admin access.
- Responsive UI for smooth usage across devices.
- Efficient request handling using FIFO mechanism.
- Optimized car selection based on price and availability.
- Structured backend with REST API integration.

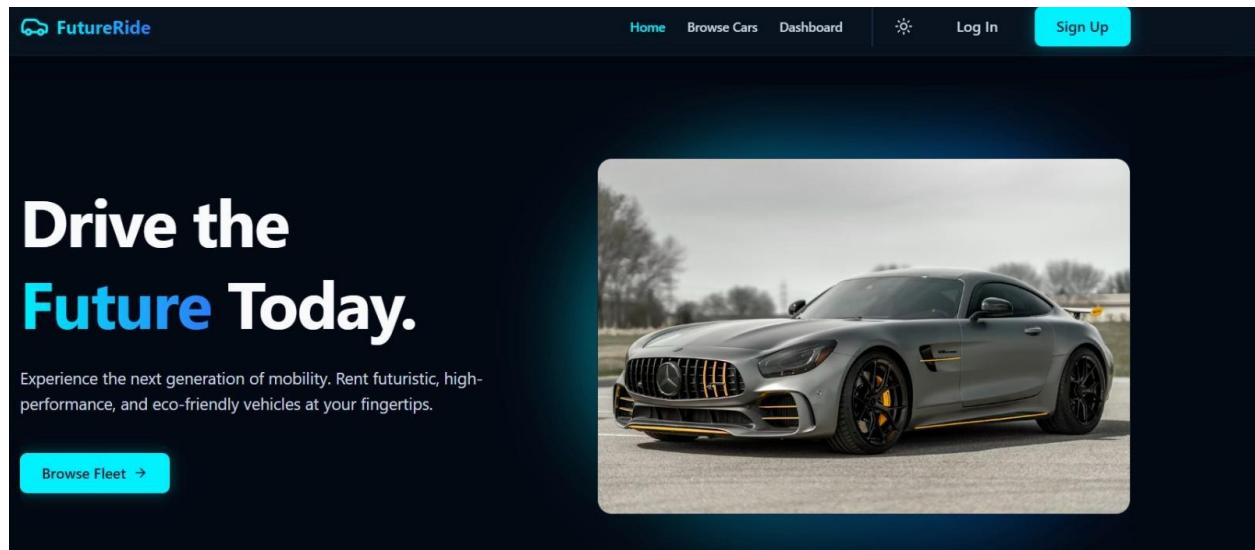
Advantages

- Efficient handling of multiple booking requests.
- Optimal car allocation using Greedy Algorithm.
- Reduced chances of booking conflicts and delays.
- Easy error correction with undo functionality.
- Improved user experience with interactive interface.
- Better resource utilization and management.
- Scalable system design for future growth.
- Clean and organized data flow between frontend and backend.

Future Enhancements

- AI-based recommendations for personalized car suggestions.
- Payment gateway integration for seamless transactions.
- Live GPS tracking for real-time vehicle location.
- Mobile app version for better accessibility.
- Advanced filtering options (car type, fuel, ratings, etc.).
- Improved Greedy Algorithm with more factors (ratings, distance, demand).
- Role-based admin controls for better system management.

DEMO



DEMO LINK

https://drive.google.com/file/d/1az3ARmV4kKpt3n4NvjfgaO123dQmwtf/f/view?usp=drive_link

GITHUB LINK : github.com/himasri255-ux



Welcome Back

Enter your credentials to access your account.

Email Address

 you@example.com

Password

[Forgot password?](#)



Sign In →

Don't have an account? [Sign up](#)

OR

 Continue with Google